A Technical Introduction to Layer normalization

Section 1

Iterate x ■ 1 ■ W x ■ 2 W x {\displaystyle x\mapsto {\frac {1}{\|Wx\|_{2}}}Wx} to convergence x ∗ {\displaystyle x^{*}} . This is the eigenvector of W {\displaystyle W} with eigenvalue ■ W ■ s {\displaystyle \|W\|_{s}} .

Section 2

x h , w , c (l) = ∑ h ′ , w ′ , c ′ K h ′ – h , w ′ – w , c , c ′ (l) x h ′ , w ′ , c ′ (l – 1) + b c (l) {\displaystyle x_{h,w,c}^{(l)}=\sum _{h',w',c'}K_{h'-h,w'-w,c,c'}^{(l)}x_{h',w',c'}^{(l-1)}+b_{c}^{(l)}}

Section 3

b x , y i = a x , y i (k + α ∑ j = max (0 , i – n / 2) min (N – 1 , i + n / 2) (a x , y j) 2) β {\displaystyle b_{x,y}^{i}={\frac {a_{x,y}^{i}}{\left(k+\alpha \sum _{j=\max(0,i-n/2)}^{\min(N-1,i+n/2)}\left(a_{x,y}^{j}\right)^{2}\right)^{\beta }}}}

Section 4

μ (t) = 1 D ∑ i = 1 D x i (t) , (σ (t)) 2 = 1 D ∑ i = 1 D (x i (t) – μ (t)) 2 {\displaystyle \mu ^{(t)}={\frac {1}{D}}\sum _{i=1}^{D}x_{i}^{(t)},\quad (\sigma ^{(t)})^{2}={\frac {1}{D}}\sum _{i=1}^{D}(x_{i}^{(t)}-\mu ^{(t)})^{2}}

Section 5

Notice that the batch index b {\displaystyle b} is removed, while the channel index c {\displaystyle c} is added. In recurrent neural networks and transformers, LayerNorm is applied individually to each timestep. For example, if the hidden vector in an RNN at timestep t {\displaystyle t} is x (t) ∈ R D {\displaystyle x^{(t)}\in \mathbb {R} ^{D}} , where D {\displaystyle D} is the dimension of the hidden vector, then LayerNorm will be applied with:

Section 6

increase the speed of training convergence, reduce sensitivity to variations and feature scales in input data, reduce overfitting, and produce better model generalization to unseen data. Normalization techniques are often theoretically justified as reducing covariance shift, smoothing optimization landscapes, and increasing regularization, though they are mainly justified by empirical success.

Section 7

x i ^ = x i – μ σ 2 + ■ , y i = γ i x i ^ + β i {\displaystyle {\hat {x_{i}}}={\frac {x_{i}-\mu }{\sqrt {\sigma ^{2}+\epsilon }}},\quad y_{i}=\gamma _{i}{\hat {x_{i}}}+\beta _{i}}

Section 8

μ c (l) = 1 H W ∑ h = 1 H ∑ w = 1 W x h , w , c (l) (σ c (l)) 2 = 1 H W ∑ h = 1 H ∑ w = 1 W (x h , w , c (l) – μ c (l)) 2 x ^ h , w , c (l) = x ^ h , w , c (l) – μ c (l) (σ c (l)) 2 + ■ y h , w , c (l) = γ c (l) x ^ h , w , c (l) + β c (l) {\displaystyle {\begin{aligned}\mu _{c}^{(l)}&={\frac {1}{HW}}\sum _{h=1}^{H}\sum _{w=1}^{W}x_{h,w,c}^{(l)}\\(\sigma _{c}^{(l)})^{2}&={\frac {1}{HW}}\sum _{h=1}^{H}\sum _{w=1}^{W}(x_{h,w,c}^{(l)}-\mu _{c}^{(l)})^{2}\\{\hat {x}}_{h,w,c}^{(l)}&={\frac {{\hat {x}}_{h,w,c}^{(l)}-\mu _{c}^{(l)}}{\sqrt {(\sigma _{c}^{(l)})^{2}+\epsilon }}}\\y_{h,w,c}^{(l)}&=\gamma _{c}^{(l)}{\hat {x}}_{h,w,c}^{(l)}+\beta _{c}^{(l)}\end{aligned}}}

Section 9

x i ^ (t) = x i (t) – μ (t) (σ (t)) 2 + ■ , y i (t) = γ i x i ^ (t) + β i {\displaystyle {\hat {x_{i}}}^{(t)}={\frac {x_{i}^{(t)}-\mu ^{(t)}}{\sqrt {(\sigma ^{(t)})^{2}+\epsilon }}},\quad y_{i}^{(t)}=\gamma _{i}{\hat {x_{i}}}^{(t)}+\beta _{i}}

Section 10

The BatchNorm module does not operate over individual inputs. Instead, it must operate over one batch of inputs at a time. Concretely, suppose we have a batch of inputs x (1) (0) , x (2) (0) , … , x (B) (0) {\displaystyle x_{(1)}^{(0)},x_{(2)}^{(0)},\dots ,x_{(B)}^{(0)}} , fed all at once into the network. We would obtain in the middle of the network some vectors:

Section 11

y (b) , i (l) = γ i x ^ (b) , i (l) + β i {\displaystyle y_{(b),i}^{(l)}=\gamma _{i}{\hat {x}}_{(b),i}^{(l)}+\beta _{i}}

Section 12

Transformers Some normalization methods were designed for use in transformers. The original 2017 transformer used the "post-LN" configuration for its LayerNorms. It was difficult to train, and required careful hyperparameter tuning and a "warm-up" in learning rate, where it starts small and gradually increases. The pre-LN convention, proposed several times in 2018, was found to be easier to train, requiring no warm-up, leading to faster convergence. FixNorm and ScaleNorm both normalize activation vectors in a transformer. The FixNorm method divides the output vectors from a transformer by their L2 norms, then multiplies by a learned parameter g {\displaystyle g} . The ScaleNorm replaces all LayerNorms inside a transformer by division with L2 norm, then multiplying by a learned parameter g ′ {\displaystyle g'} (shared by all ScaleNorm modules of a transformer). Query-Key normalization (QKNorm) normalizes query and key vectors to have unit L2 norm. In nGPT, many vectors are normalized to have unit L2 norm: hidden state vectors, input and output embedding vectors, weight matrix columns, and query and key vectors.

Section 13

μ (l) = 1 H W C ∑ h = 1 H ∑ w = 1 W ∑ c = 1 C x h , w , c (l) (σ (l)) 2 = 1 H W C ∑ h = 1 H ∑ w = 1 W ∑ c = 1 C (x h , w , c (l) – μ (l)) 2 x ^ h , w , c (l) = x ^ h , w , c (l) – μ (l) (σ (l)) 2 + ■ y h , w , c (l) = γ (l) x ^ h , w , c (l) + β (l) {\displaystyle {\begin{aligned}\mu ^{(l)}&={\frac {1}{HWC}}\sum _{h=1}^{H}\sum _{w=1}^{W}\sum _{c=1}^{C}x_{h,w,c}^{(l)}\\\sigma ^{(l)})^{2}&={\frac {1}{HWC}}\sum _{h=1}^{H}\sum _{w=1}^{W}\sum _{c=1}^{C}(x_{h,w,c}^{(l)}-\mu ^{(l)})^{2}\\{\hat {x}}_{h,w,c}^{(l)}&={\frac {{\hat {x}}_{h,w,c}^{(l)}-\mu ^{(l)}}{\sqrt {(\sigma ^{(l)})^{2}+\epsilon }}}\\y_{h,w,c}^{(l)}&=\gamma ^{(l)}{\hat {x}}_{h,w,c}^{(l)}+\beta ^{(l)}\end{aligned}}}